23.04.2026

OPENCAMPUS.sh

Applied Machine Learning

INTRODUCTION TO PYTORCH



Julia McCoy
@JuliaEMcCoy

We are sending our kids to school to memorize facts that AI can retrieve in 0.3 seconds.

We're grading them on essays that AI writes better than their teachers.
...



Will Bredderman
@WillBredderman

Worst of all, gym classes are making them run miles an Uber could cover in less than a minute.

https://x.com/JuliaEMcCoy/status/2028538251196371374
https://x.com/WillBredderman/status/2028861498651537828/

- Quiz
- Assignment Presentation & Recap
- Projects
- Model Design & Training
- Activation Functions: ReLU
- [Neural Networks in Detail (some math)]
- Outlook: Next Week

QUIZ



https://forms.office.com/r/kYhxk6JvjN

ASSIGNMENT PRESENTATION

ASSIGNMENT RECAP

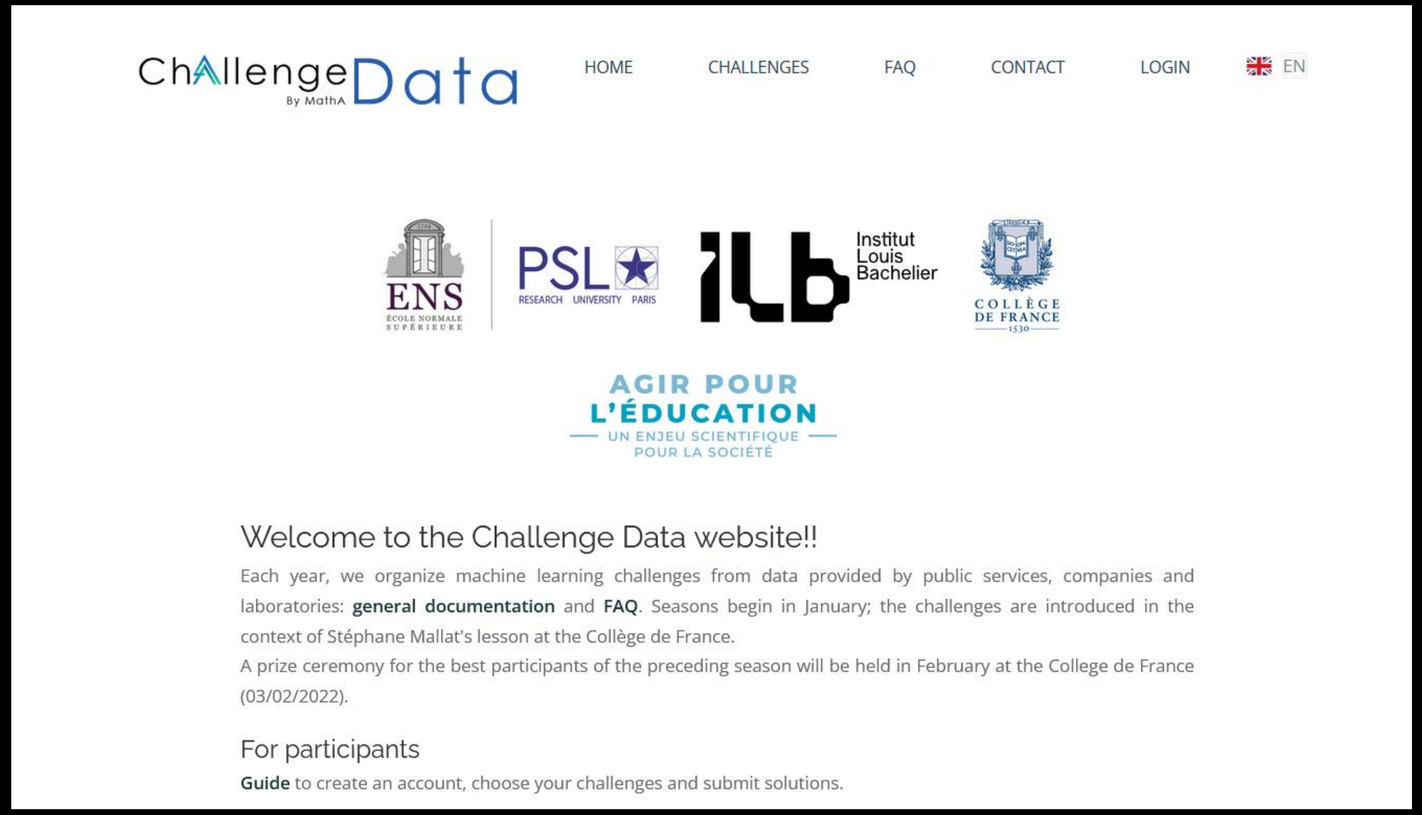
ASSIGNMENT RECAP

- Data Handling, Pre-processing & Tensor Reshaping
- Feature Engineering
- Handling of Temporal Data
- Model Training
- Exploding Gradients 
  - Importance of Monitoring
  - Importance of Scaling for stable Gradients and efficient Training
- Scaling of Features and Targets

PROJECTS

PROJECTS

- Bring your own idea and data
- Choose a project from the Challenge Data platform



PROJECT MILESTONES

23.04.	Present your Ideas
30.04.	Form Groups & Put your Name in the Excel
07.05.	Literature Review
14.05.	Dataset Characteristics and Individual Feedback Sessions (**PUBLIC HOLIDAY**)

Deadline for completing the repo sections: 17.05.

21.05.	Definition of Model Evaluation
28.05.	Project Work
04.06.	Baseline Model Estimation

Deadline for completing the repo sections: 07.06.

11.06.	Project Work / Optional Feedback Sessions (**NO REGULAR CLASS**)
25.06.	Project Presentations, Part I
02.07.	Project Presentations, Part II

Submission deadline for the documented repo: 01.08.2026

MODEL DESIGN & TRAINING

MODEL DESIGN & TRAINING: BUILDING BLOCKS

- Before we can train a model, we need to define **three components**:

MODEL DESIGN & TRAINING: BUILDING BLOCKS

- Before we can train a model, we need to define **three components**:

Model

Defines the layers and how the data flows forward (**forward pass**)

```
import torch.nn as nn

C = 2         # Number of input features (channels)
L = 1         # Dimension of output features (labels)

model = nn.Linear(
    in_features=C,
    out_features=L
  )
```

MODEL DESIGN & TRAINING: BUILDING BLOCKS

- Before we can train a model, we need to define **three components**:

Model

Defines the layers and how the data flows forward (**forward pass**)

```
import torch.nn as nn

C = 2         # Number of input features (channels)
L = 1         # Dimension of output features (labels)

model = nn.Linear(
    in_features=C,
    out_features=L
  )
```

Loss Function

Measures how far the **predictions** are from the **true target values**

```
import torch.nn as nn

loss_fn = nn.MSELoss()
# Call as `loss_fn(predictions, true_targets)`
```

MODEL DESIGN & TRAINING: BUILDING BLOCKS

- Before we can train a model, we need to define **three components**:

Model

Defines the layers and how the data flows forward (**forward pass**)

```python
import torch.nn as nn

C = 2        # Number of input features (channels)
L = 1        # Dimension of output features (labels)

model = nn.Linear(
    in_features=C,
    out_features=L
  )
```

Loss Function

Measures how far the **predictions** are from the **true target values**

```python
import torch.nn as nn

loss_fn = nn.MSELoss()
# Call as `loss_fn(predictions, true_targets)`
```

Optimizer

Defines **update rule** for parameters of the model and adjusts them based on gradients of loss function

```python
import torch.optim as optim

optimizer = optim.SGD(
    model.parameters(),
    lr=0.01
  )
```

MODEL DESIGN & TRAINING: TRAINING STEPS

- After defining the three components, we can train a model by following the required **five steps**:

MODEL DESIGN & TRAINING: TRAINING STEPS

- After defining the three components, we can train a model by following the required **five steps**:

1. **Zero Gradients**
   Reset gradients from the previous epoch

```
N_EPOCHS = 10

for epoch in range(N_EPOCHS):
  # 1. Zero Gradients
  optimizer.zero_grad()
```

MODEL DESIGN & TRAINING: TRAINING STEPS

- After defining the three components, we can train a model by following the required **five steps**:

1. **Zero Gradients**
   Reset gradients from the previous epoch
2. **Forward Pass**
   Run inputs through the model to get predictions

```
N_EPOCHS = 10

for epoch in range(N_EPOCHS):
  # 1. Zero Gradients
  optimizer.zero_grad()

  # 2. Forward Pass
  predictions = model(x)
```

MODEL DESIGN & TRAINING: TRAINING STEPS

- After defining the three components, we can train a model by following the required **five steps**:

1. **Zero Gradients**
   Reset gradients from the previous epoch
2. **Forward Pass**
   Run inputs through the model to get predictions
3. **Compute Loss**
   Measure how wrong the predictions are

```
N_EPOCHS = 10

for epoch in range(N_EPOCHS):
  # 1. Zero Gradients
  optimizer.zero_grad()

  # 2. Forward Pass
  predictions = model(x)

  # 3. Compute Loss
  loss = loss_fn(predictions, y)
```

MODEL DESIGN & TRAINING: TRAINING STEPS

- After defining the three components, we can train a model by following the required **five steps**:

1. **Zero Gradients**
   Reset gradients from the previous epoch
2. **Forward Pass**
   Run inputs through the model to get predictions
3. **Compute Loss**
   Measure how wrong the predictions are
4. **Backward Pass**
   Compute the gradients $\partial \mathcal{L}/\partial w$ for every parameter

```
N_EPOCHS = 10

for epoch in range(N_EPOCHS):
  # 1. Zero Gradients
  optimizer.zero_grad()

  # 2. Forward Pass
  predictions = model(x)

  # 3. Compute Loss
  loss = loss_fn(predictions, y)

  # 4. Backward Pass
  loss.backward()
```

MODEL DESIGN & TRAINING: TRAINING STEPS

- After defining the three components, we can train a model by following the required **five steps**:

1. **Zero Gradients**
   Reset gradients from the previous epoch
2. **Forward Pass**
   Run inputs through the model to get predictions
3. **Compute Loss**
   Measure how wrong the predictions are
4. **Backward Pass**
   Compute the gradients $\partial\mathcal{L}/\partial w$ for every parameter
5. **Update Model Parameters**
   Apply the gradient-based update rule of the optimizer

```
N_EPOCHS = 10

for epoch in range(N_EPOCHS):
  # 1. Zero Gradients
  optimizer.zero_grad()

  # 2. Forward Pass
  predictions = model(x)

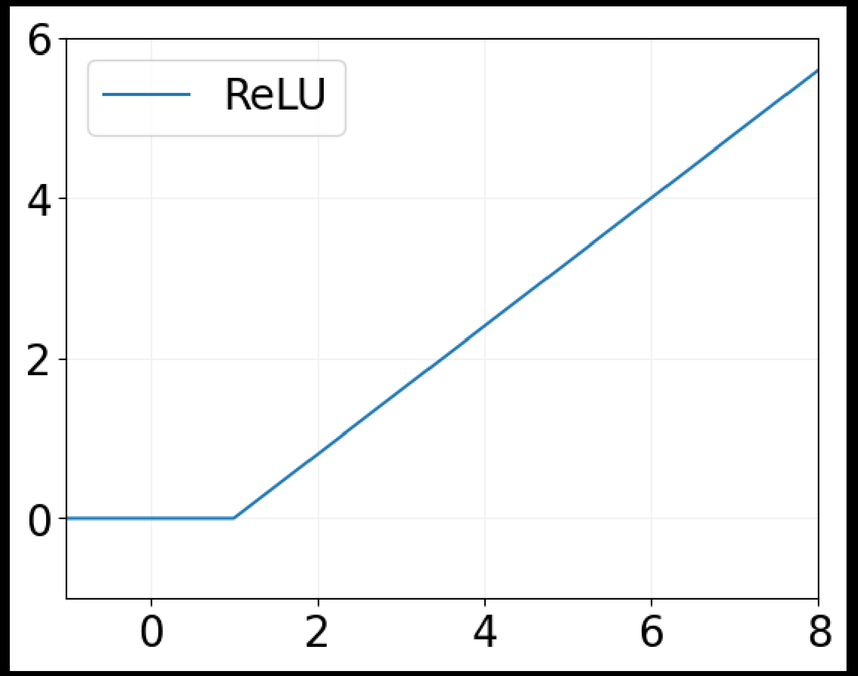
  # 3. Compute Loss
  loss = loss_fn(predictions, y)

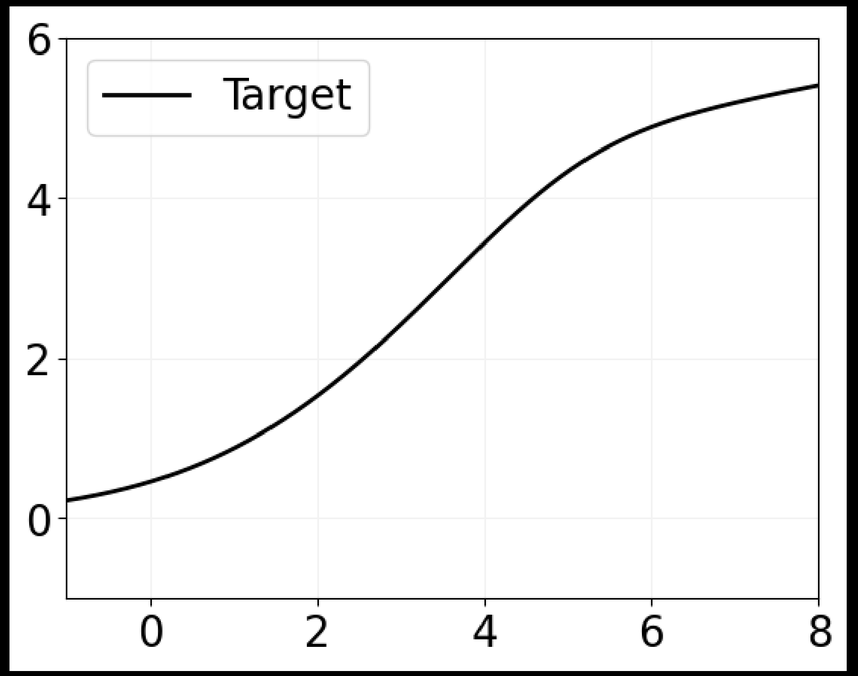
  # 4. Backward Pass
  loss.backward()

  # 5. Update Model Parameters
  optimizer.step()
```

ACTIVATION FUNCTIONS: RELU

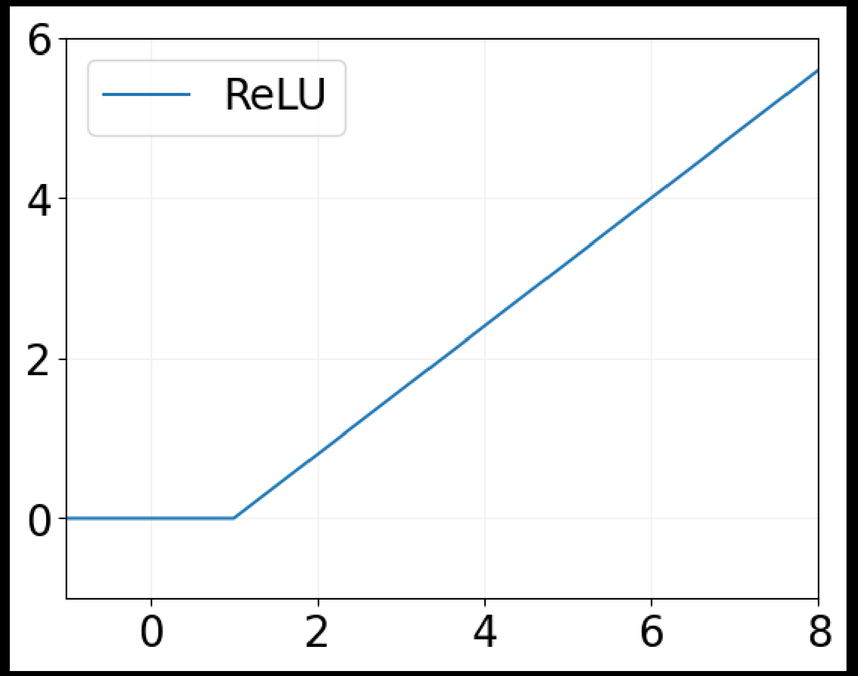
ACTIVATION FUNCTIONS: RELU

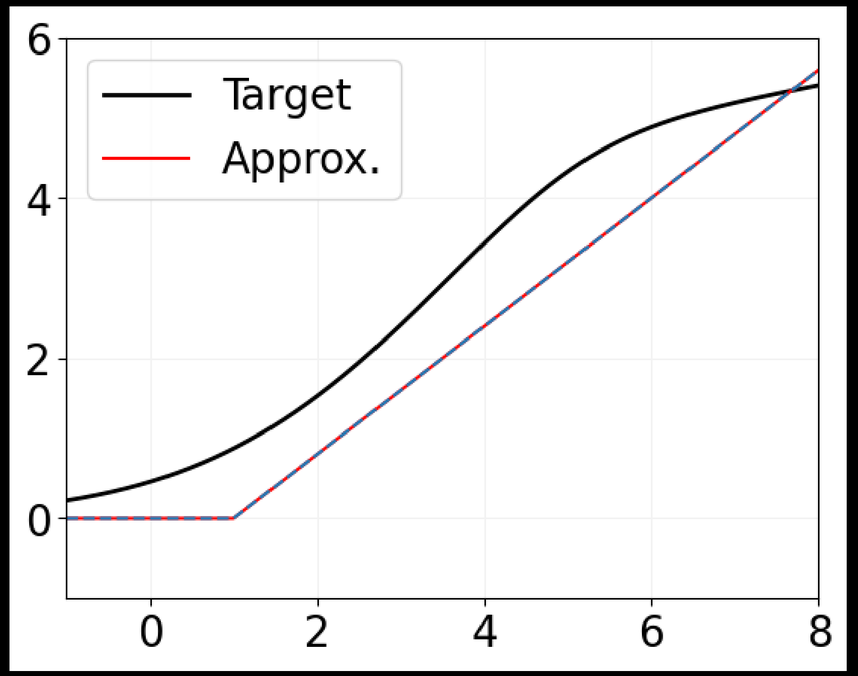




- How does the **Rectified Linear Unit** (**ReLU**) activation function help us to approximate non-linear data?

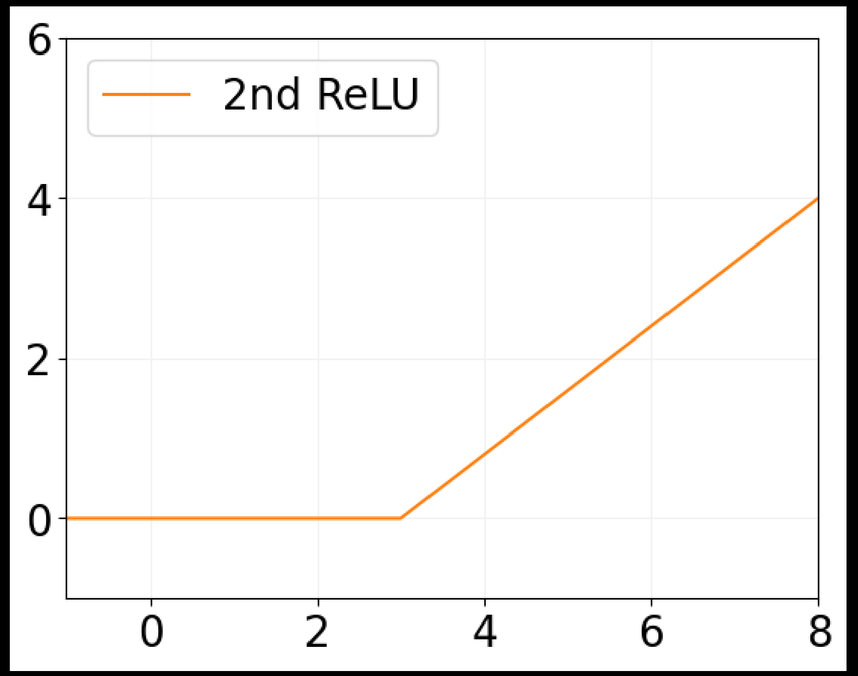
ACTIVATION FUNCTIONS: RELU

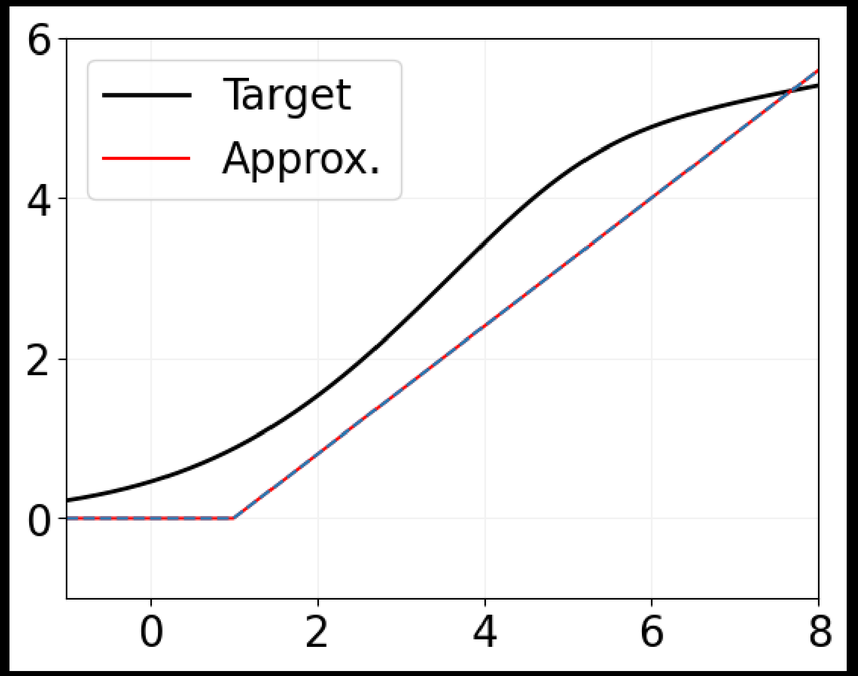




- How does the **Rectified Linear Unit** (**ReLU**) activation function help us to approximate non-linear data?

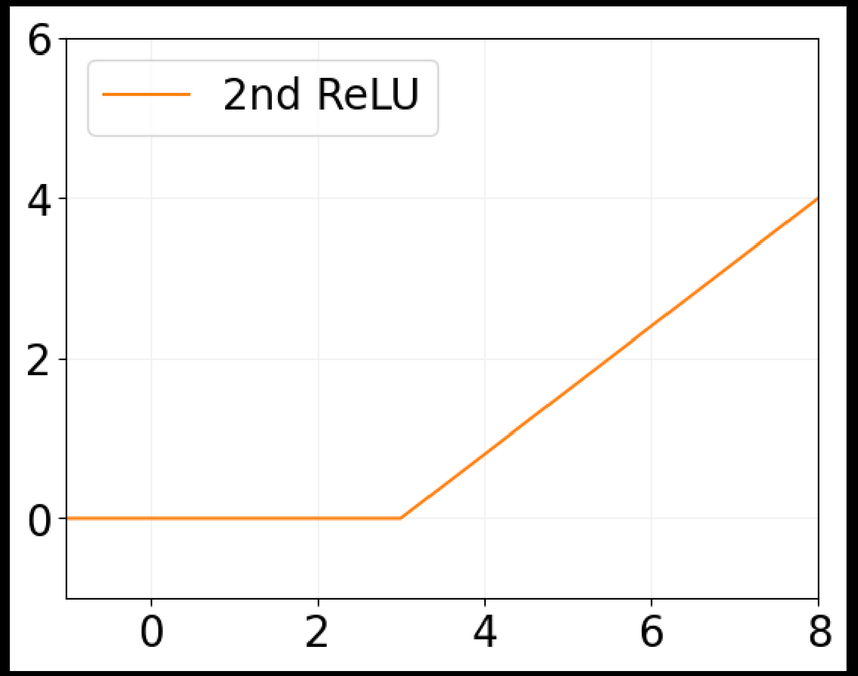
ACTIVATION FUNCTIONS: RELU

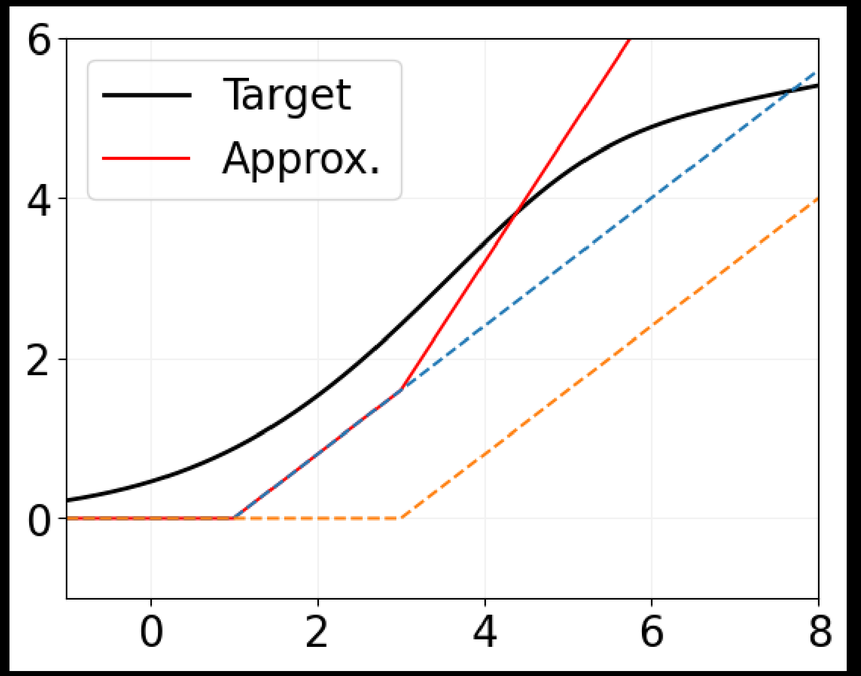




- How does the **Rectified Linear Unit** (**ReLU**) activation function help us to approximate non-linear data?

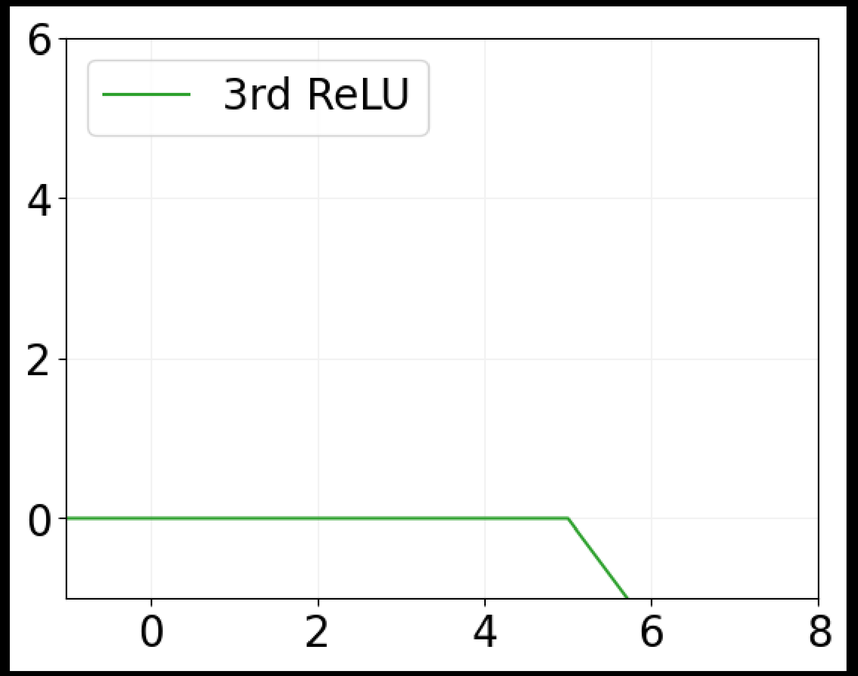
ACTIVATION FUNCTIONS: RELU

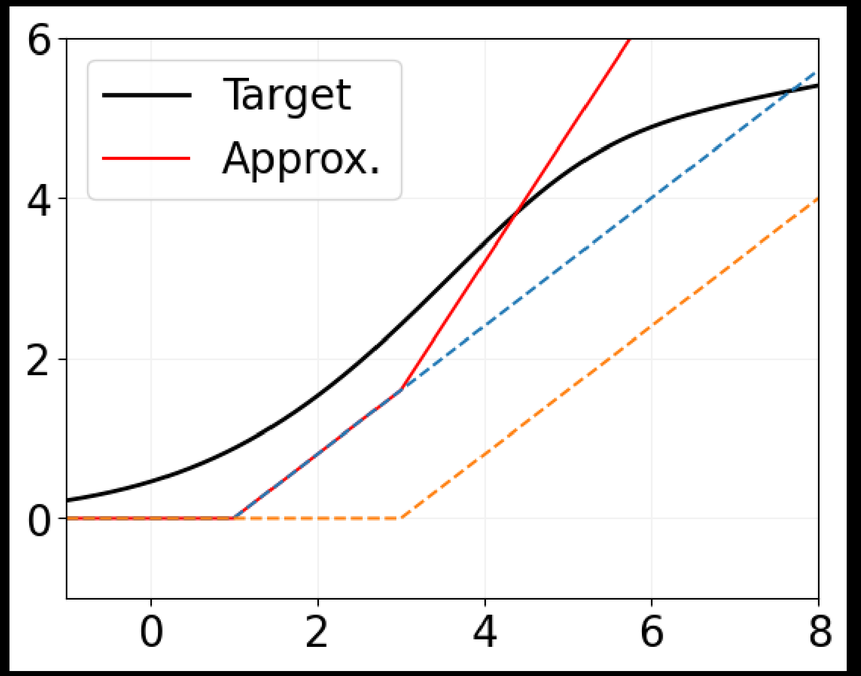




- How does the **Rectified Linear Unit** (**ReLU**) activation function help us to approximate non-linear data?

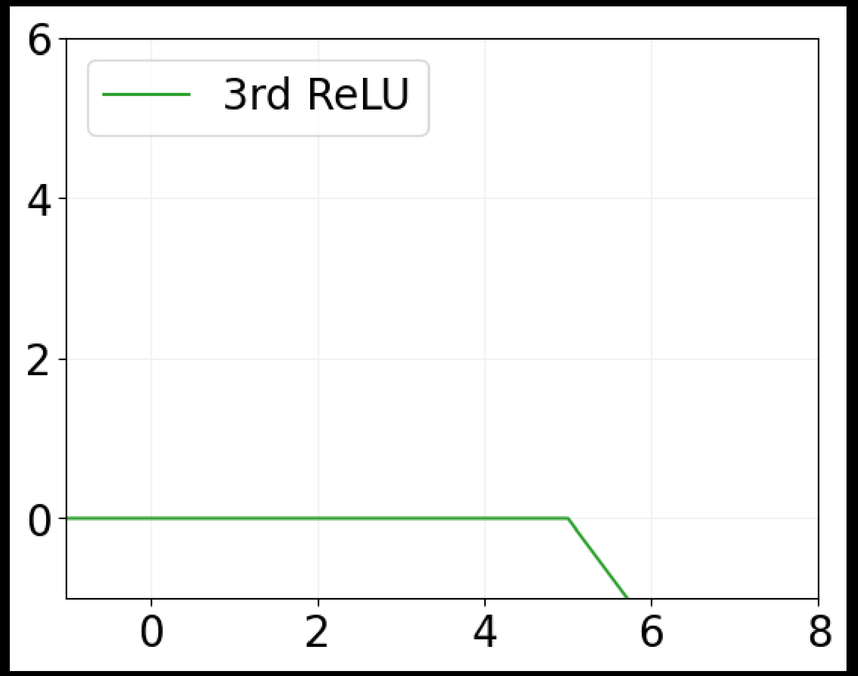
ACTIVATION FUNCTIONS: RELU

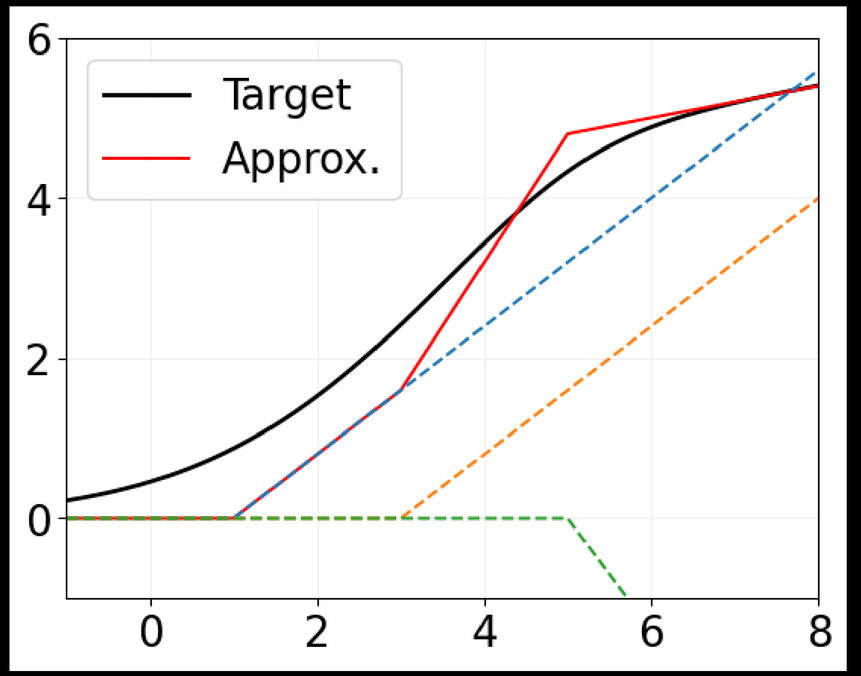




- How does the **Rectified Linear Unit** (**ReLU**) activation function help us to approximate non-linear data?

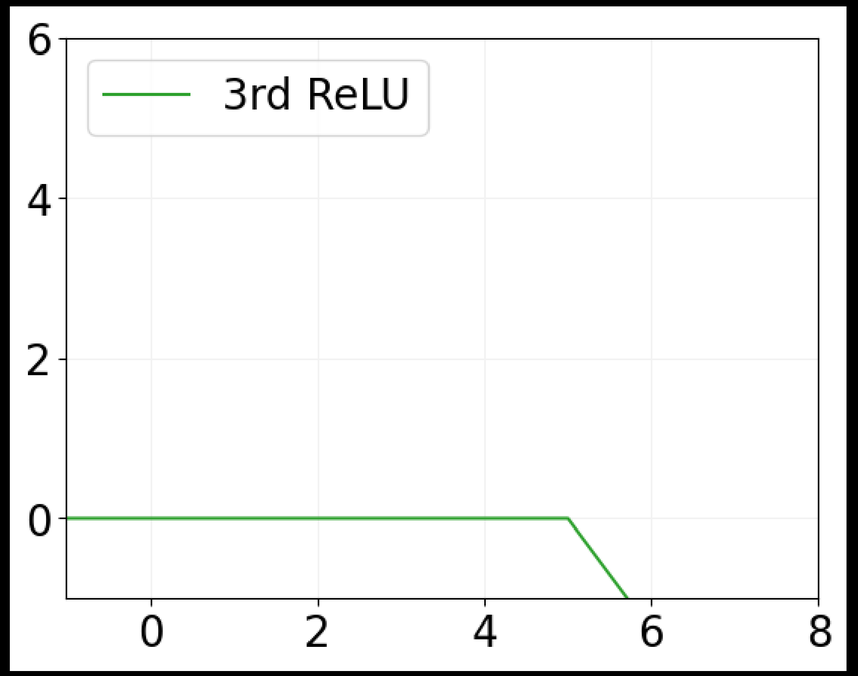
ACTIVATION FUNCTIONS: RELU

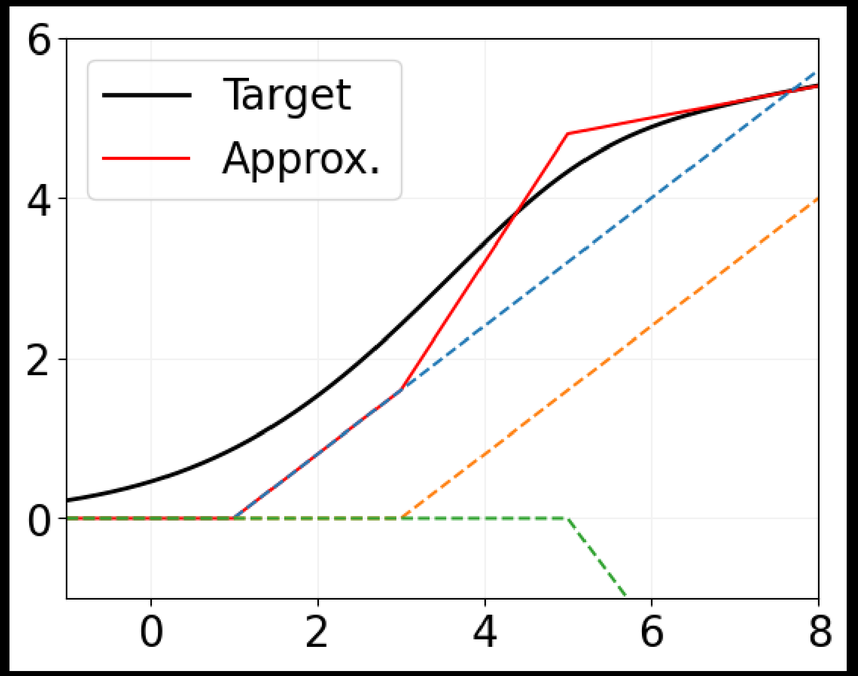




- How does the **Rectified Linear Unit** (**ReLU**) activation function help us to approximate non-linear data?

ACTIVATION FUNCTIONS: RELU





- How does the **Rectified Linear Unit** (**ReLU**) activation function help us to approximate non-linear data?

  → **The final `nn.Linear()` layer learns to scale and combine the ReLU activations from the previous hidden layer**

NEURAL NETWORKS IN DETAIL

WHAT IS A NEURAL NETWORK?

WHAT IS A NEURAL NETWORK?

- A NN is an approximation $\hat{y} = f(x, w, b)$ of the **true target values** y using **input features** $x = x_1, \dots, x_C$ denoted as the **pair** $P = (x, y)$

WHAT IS A NEURAL NETWORK?

- A NN is an approximation $\hat{y} = f(x, w, b)$ of the **true target values** y using **input features** $x = x_1, \dots, x_C$ denoted as the **pair** $P = (x, y)$
- Usually, when training a model, we use a series of N **pairs** $P^{(1)}, \dots, P^{(N)}$ written as $\left[P^{(1)}, \dots, P^{(N)}\right] = \left[\left(x^{(1)}, y^{(1)}\right), \dots, \left(x^{(N)}, y^{(N)}\right)\right]$

WHAT IS A NEURAL NETWORK?

- A NN is an approximation $\hat{y} = f(x, w, b)$ of the **true target values** y using **input features** $x = x_1, \dots, x_C$ denoted as the **pair** $P = (x, y)$
- Usually, when training a model, we use a series of N **pairs** $P^{(1)}, \dots, P^{(N)}$ written as $\left[P^{(1)}, \dots, P^{(N)}\right] = \left[\left(x^{(1)}, y^{(1)}\right), \dots, \left(x^{(N)}, y^{(N)}\right)\right]$
- We want to learn the **weights** w and **biases** b such that $\hat{y} \approx y$

WHAT IS A NEURAL NETWORK?

- A NN is an approximation $\hat{y} = f(x, w, b)$ of the **true target values** y using **input features** $x = x_1, \dots, x_C$ denoted as the **pair** $P = (x, y)$
- Usually, when training a model, we use a series of N **pairs** $P^{(1)}, \dots, P^{(N)}$ written as $\left[P^{(1)}, \dots, P^{(N)}\right] = \left[\left(x^{(1)}, y^{(1)}\right), \dots, \left(x^{(N)}, y^{(N)}\right)\right]$
- We want to learn the **weights** w and **biases** b such that $\hat{y} \approx y$

Example: (dropping the **pair dimension** for simplicity)

- Assume that we have **two input features** ($C = 2$)
- The **true target values** may have **one output dimension** ($L = 1$)
- This simple linear model is $\hat{y}_1 = f(x, w, b) = x_1 \cdot w_1 + x_2 \cdot w_2 + b$

A NEURAL NETWORK IS $\hat{y} = f(x, w, b)$

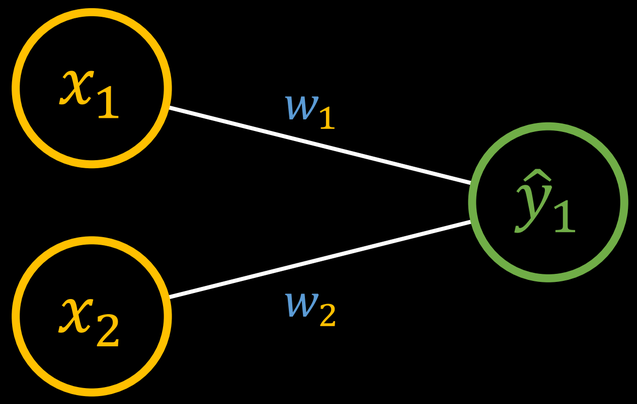
- Simple linear model with **two input features** and **one output dimension**

$$\hat{y}_1 = f(x, w, b) = x_1 \cdot w_1 + x_2 \cdot w_2 + b$$

A NEURAL NETWORK IS $\hat{y} = f(x, w, b)$

- Simple linear model with **two input features** and **one output dimension**

$$\hat{y}_1 = f(x, w, b) = x_1 \cdot w_1 + x_2 \cdot w_2 + b$$

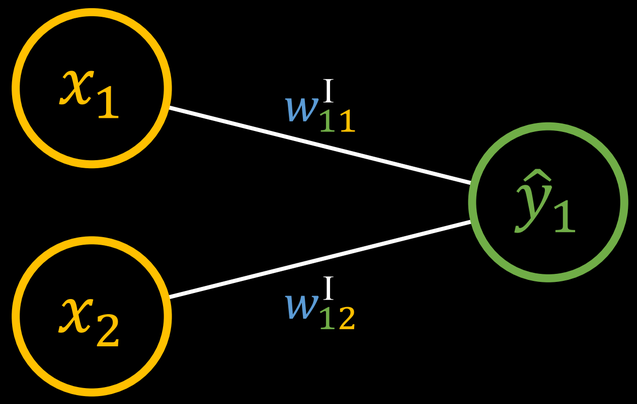


The diagram ignores the **bias** b for simplicity

A NEURAL NETWORK IS $\hat{y} = f(x, w, b)$

- Simple linear model with **two input features** and **one output dimension**

$$\hat{y}_1 = f(x, w, b) = x_1 \cdot w^{\mathrm{I}}_{11} + x_2 \cdot w^{\mathrm{I}}_{12} + b^{\mathrm{I}}_1$$

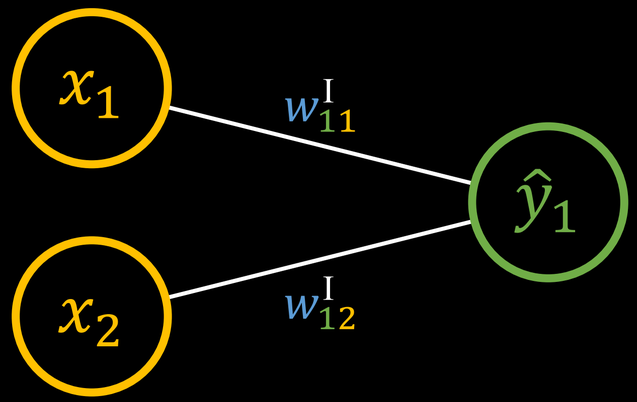


The diagram ignores the **bias** b for simplicity

A NEURAL NETWORK IS $\hat{y} = f(x, w, b)$

- Simple linear model with **two input features** and **one output dimension**

$$\hat{y}_1 = f(x, w, b) = x_1 \cdot w_{11}^{\mathrm{I}} + x_2 \cdot w_{12}^{\mathrm{I}} + b_1^{\mathrm{I}}$$



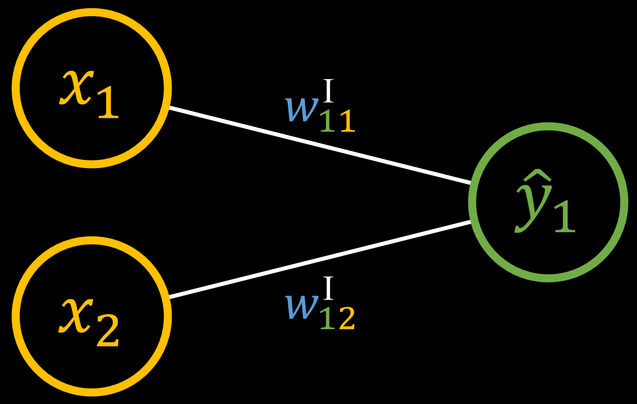
- As deeper networks have multiple layers, we need to introduce a new notation by writing w_1 as w_{11}^{I}
- w_{11}^{I} is the **weight** of the **first layer** (denoted as **layer** I) used to compute the **first output feature** $\hat{y}_1$ of **layer** I by scaling the **first input feature** x_1

The diagram ignores the **bias** b for simplicity

A NEURAL NETWORK IS $\hat{y} = f(x, w, b)$

- Simple linear model with **two input features** and **one output dimension**

$$\hat{y}_1 = f(x, w, b) = x_1 \cdot w^{\mathrm{I}}_{11} + x_2 \cdot w^{\mathrm{I}}_{12} + b^{\mathrm{I}}_1$$



- Internally, PyTorch uses matrix multiplication, so we usually write

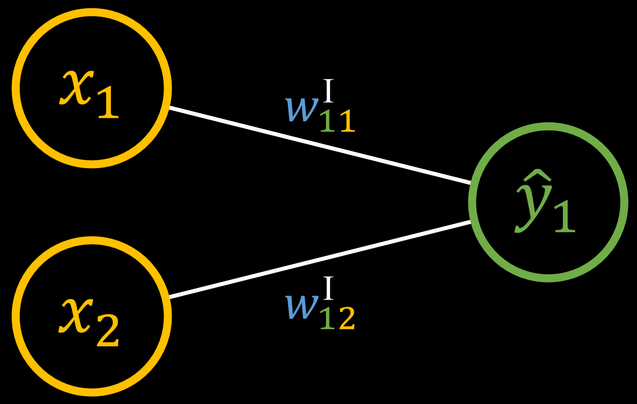
$$\hat{y}_1 = f(x, w, b) = \begin{bmatrix} x_1 & x_2 \end{bmatrix} \cdot \begin{bmatrix} w^{\mathrm{I}}_{11} & w^{\mathrm{I}}_{12} \end{bmatrix}^T + b$$

The diagram ignores the **bias** b for simplicity

A NEURAL NETWORK IS $\hat{y} = f(x, w, b)$

- Simple linear model with **two input features** and **one output dimension**

$$\hat{y}_1 = f(x, w, b) = x_1 \cdot w^{\mathrm{I}}_{11} + x_2 \cdot w^{\mathrm{I}}_{12} + b^{\mathrm{I}}_1$$



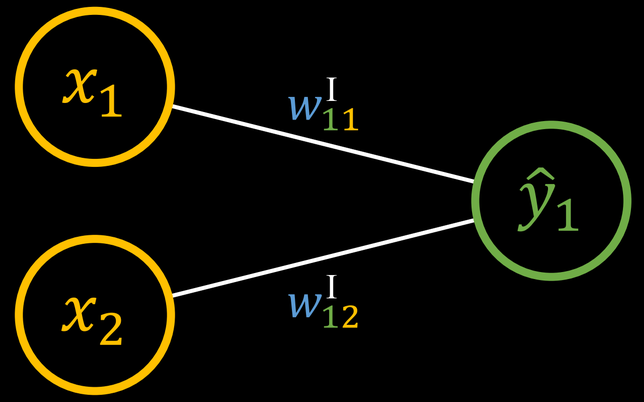
- Internally, PyTorch uses matrix multiplication, so we usually write

$$\hat{y}_1 = f(x, w, b) = \quad X \quad \cdot \quad \left(W^{\mathrm{I}}\right)^T \quad + B^{\mathrm{I}}$$

The diagram ignores the **bias** b for simplicity

A NEURAL NETWORK IS $\hat{y} = f(x, w, b)$

- PyTorch makes it easy to define this simple linear model with
  - **two input features** ($C = 2$)
  - **one output dimension** ($\mathrm{L} = 1$)



$$\hat{y}_1 = X \cdot \left(W^{\mathrm{I}}\right)^T + B^{\mathrm{I}}$$

The diagram ignores the **bias** b for simplicity

A NEURAL NETWORK IS $\hat{y} = f(x, w, b)$

- PyTorch makes it easy to define this simple linear model with
  - **two input features** $(C = 2)$
  - **one output dimension** $(\mathrm{L} = 1)$

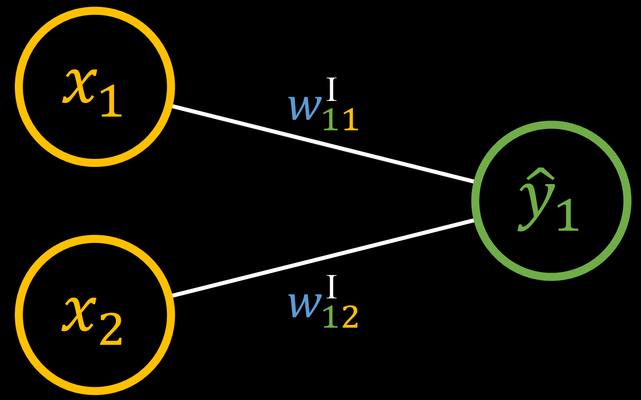
```
import torch.nn as nn

C = 2         # Number of input features (channels)
L = 1         # Dimension of output features (labels)

simple_model = nn.Linear(in_features=C, out_features=L)

print(simple_model.weight.shape)
print(simple_model.bias.shape)
```

```
torch.Size([1, 2])
torch.Size([1])
```



$$\hat{y}_1 = X \cdot \left(W^{\mathrm{I}}\right)^T + B^{\mathrm{I}}$$

The diagram ignores the **bias** b for simplicity

A NEURAL NETWORK IS $\hat{y} = f(x, w, b)$

- PyTorch makes it easy to define this simple linear model with
  - **two input features** $(C = 2)$
  - **one output dimension** $(\mathrm{L} = 1)$

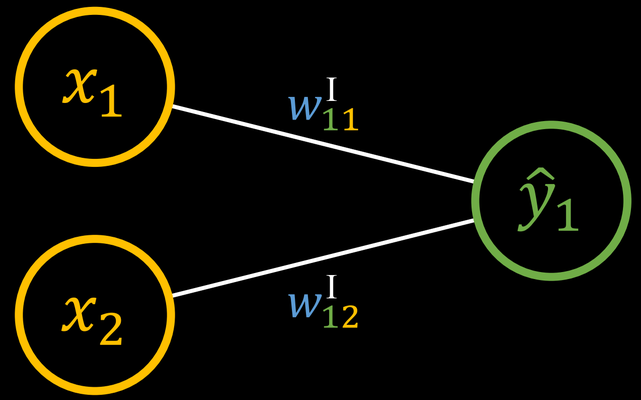
```
import torch.nn as nn

C = 2         # Number of input features (channels)
L = 1         # Dimension of output features (labels)

simple_model = nn.Linear(in_features=C, out_features=L)

print(simple_model.weight.shape)
print(simple_model.bias.shape)
```

```
torch.Size([1, 2])
torch.Size([1])
```



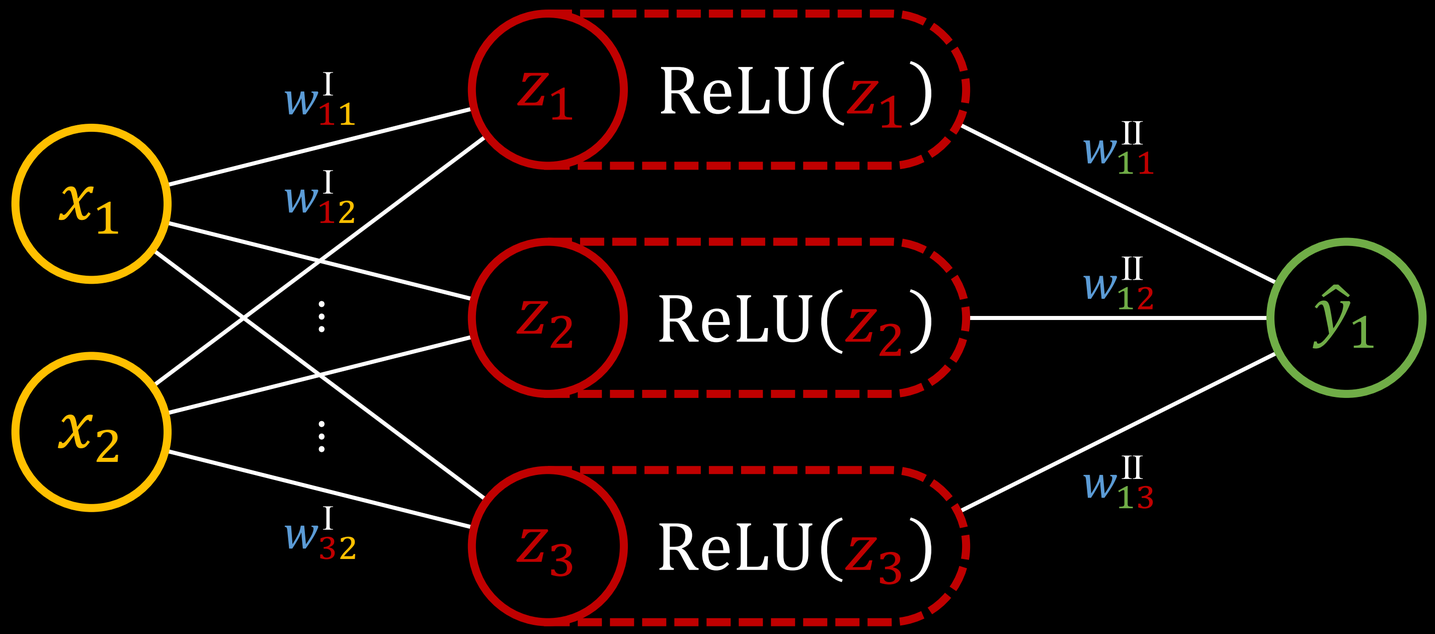
$$\hat{y}_1 = X \cdot \left(W^{\mathrm{I}}\right)^T + B^{\mathrm{I}}$$

- The matrix W^{I} has L **rows** and C **columns** corresponding to the **inputs** and **outputs**

The diagram ignores the **bias** b for simplicity

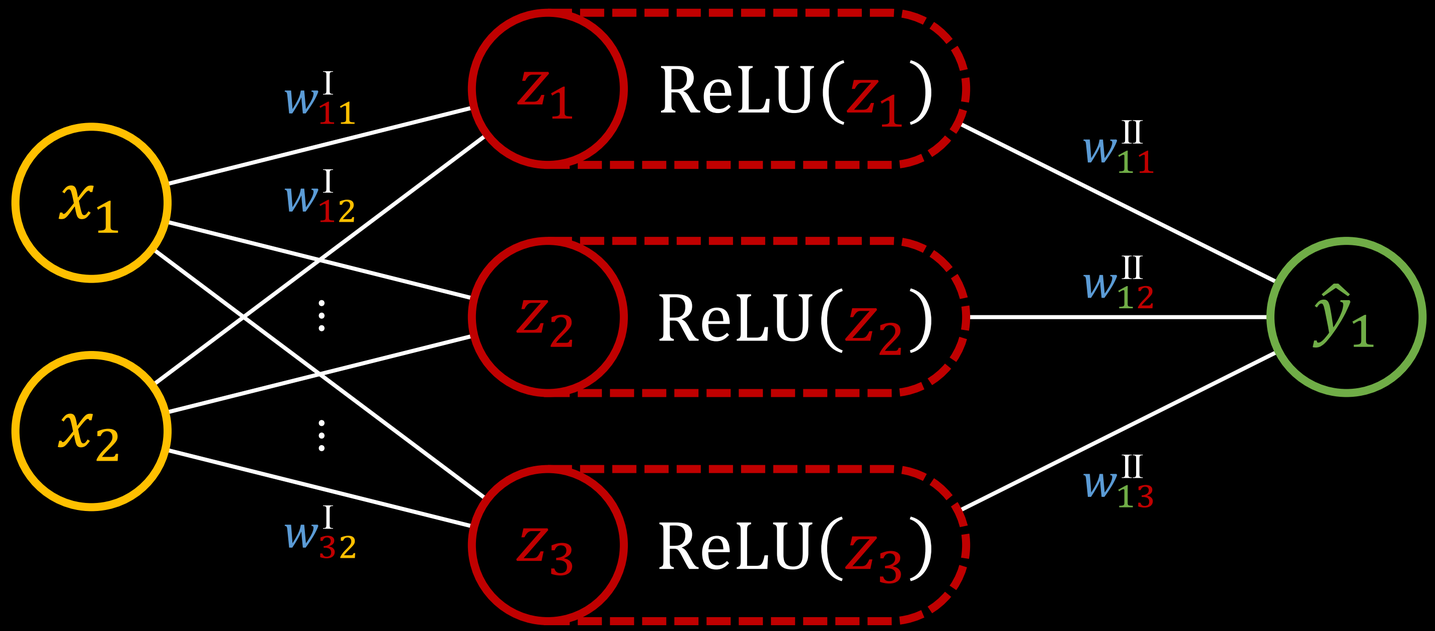
DEEP NEURAL NETWORKS

- For the model to learn non-linear relationships, we need to introduce **non-linear activation functions** such as $\mathrm{ReLU}(x) = \max(x, 0)$
- A deep neural network relies on intermediary **hidden states** $z_1, \ldots, z_H$



The diagram ignores the **bias** b for simplicity

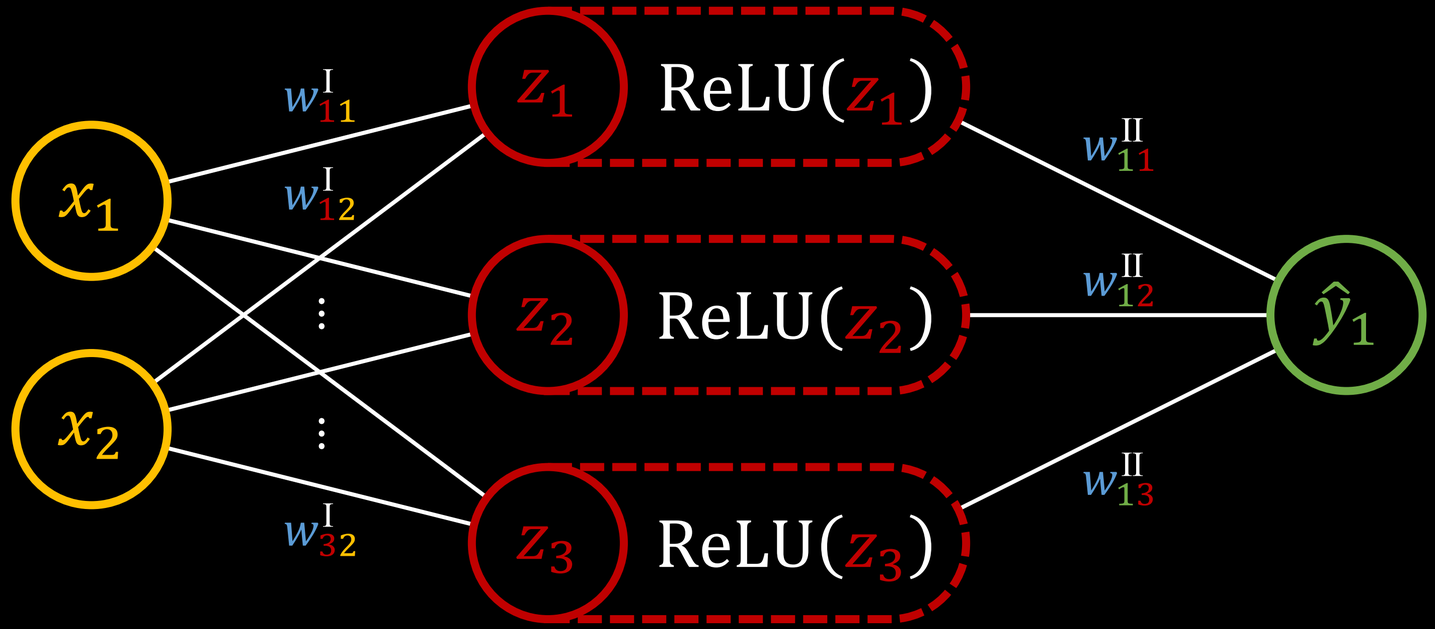
DEEP NEURAL NETWORKS



$$\hat{y}_1 = \mathrm{ReLU}\big(z_1(x)\big) \cdot w_{11}^{\mathrm{II}}$$

The diagram ignores the **bias** b for simplicity

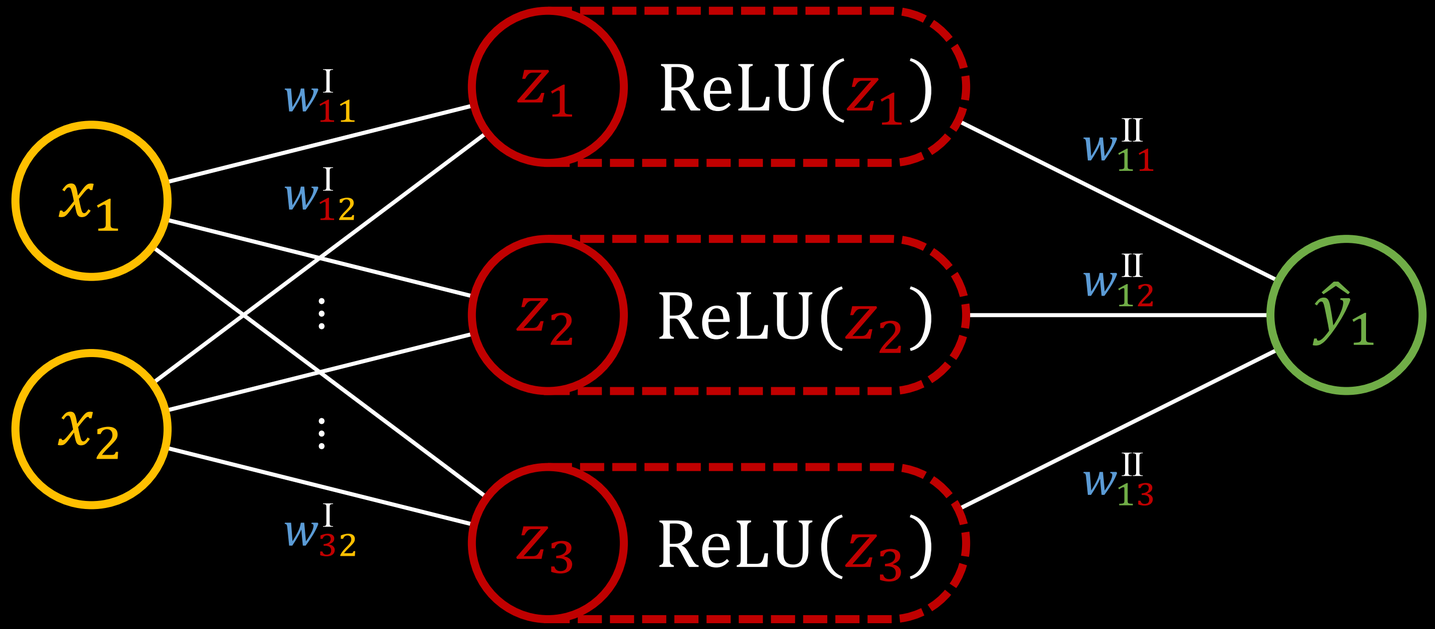
DEEP NEURAL NETWORKS



$$\hat{y}_1 = \mathrm{ReLU}\big(z_1(x)\big) \cdot w_{11}^{\mathrm{II}} + \mathrm{ReLU}\big(z_2(x)\big) \cdot w_{12}^{\mathrm{II}}$$

The diagram ignores the **bias** b for simplicity

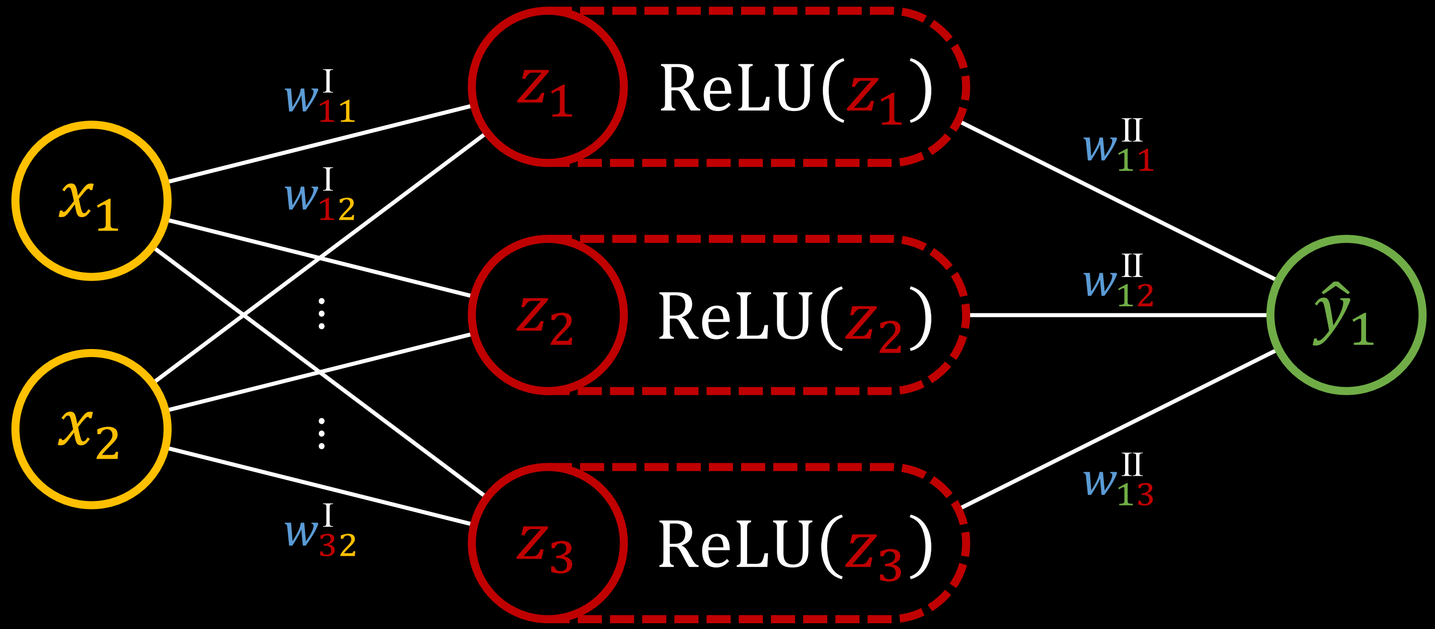
DEEP NEURAL NETWORKS



$$\hat{y}_1 = \mathrm{ReLU}\big(z_1(x)\big) \cdot w_{11}^{\mathrm{II}} + \mathrm{ReLU}\big(z_2(x)\big) \cdot w_{12}^{\mathrm{II}} + \mathrm{ReLU}\big(z_3(x)\big) \cdot w_{13}^{\mathrm{II}}$$

The diagram ignores the **bias** b for simplicity

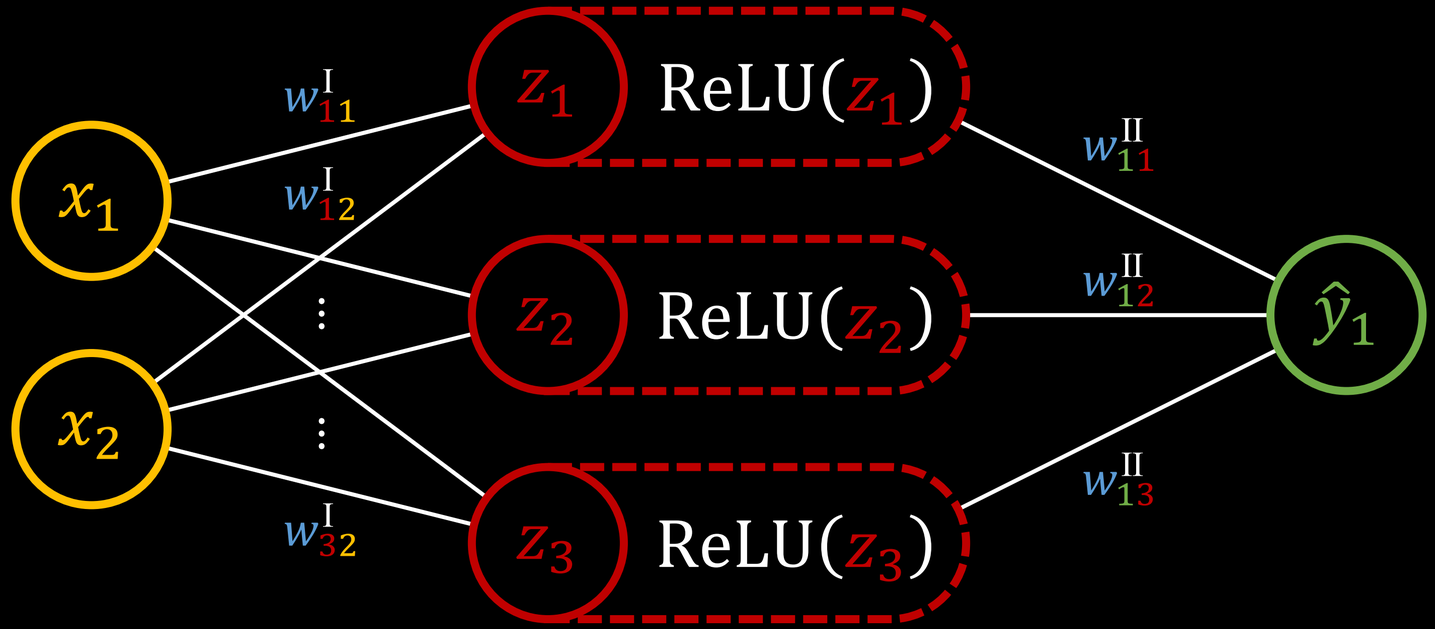
DEEP NEURAL NETWORKS



$$\hat{y}_1 = \mathrm{ReLU}\big(z_1(x)\big) \cdot w_{11}^{\mathrm{II}} + \mathrm{ReLU}\big(z_2(x)\big) \cdot w_{12}^{\mathrm{II}} + \mathrm{ReLU}\big(z_3(x)\big) \cdot w_{13}^{\mathrm{II}} + b_1^{\mathrm{II}}$$

The diagram ignores the **bias** b for simplicity

DEEP NEURAL NETWORKS

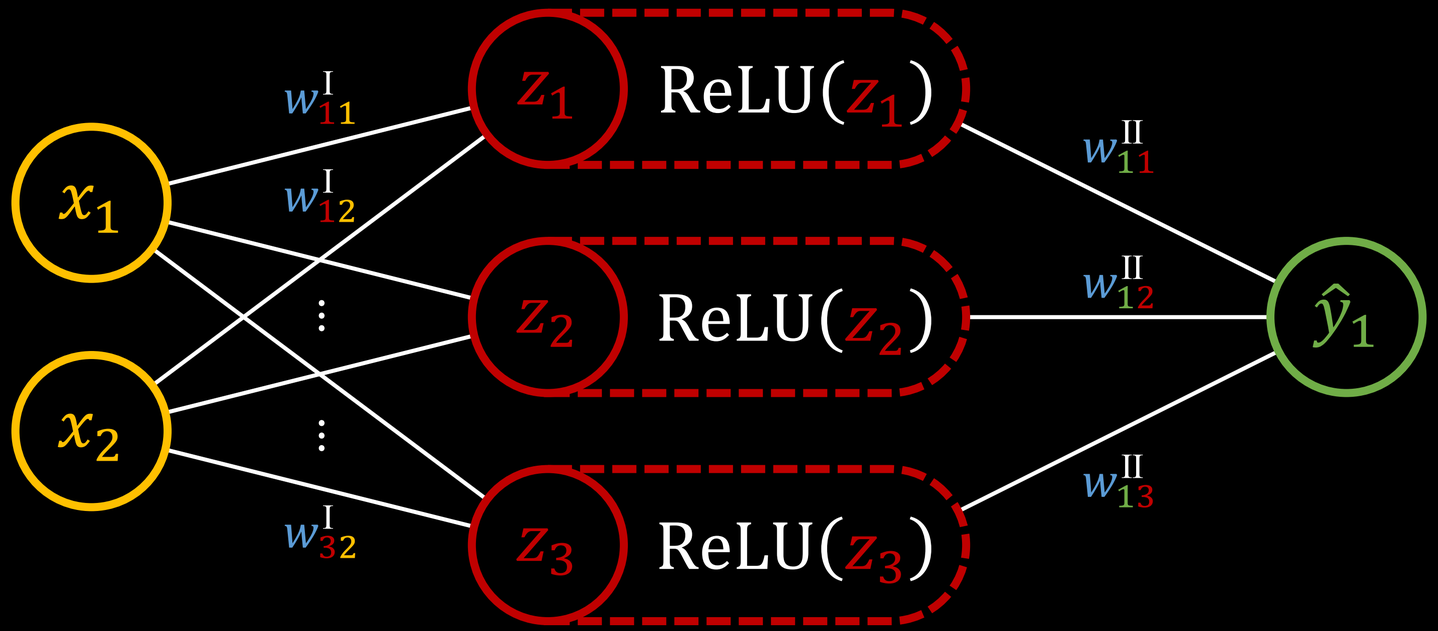


$$\hat{y}_1 = \mathrm{ReLU}\big(z_1(x)\big) \cdot w_{11}^{\mathrm{II}} + \mathrm{ReLU}\big(z_2(x)\big) \cdot w_{12}^{\mathrm{II}} + \mathrm{ReLU}\big(z_3(x)\big) \cdot w_{13}^{\mathrm{II}} + b_1^{\mathrm{II}}$$

$$= \mathrm{ReLU}(Z) \cdot \big(W^{\mathrm{II}}\big)^T + B^{\mathrm{II}}$$

The diagram ignores the **bias** b for simplicity

DEEP NEURAL NETWORKS



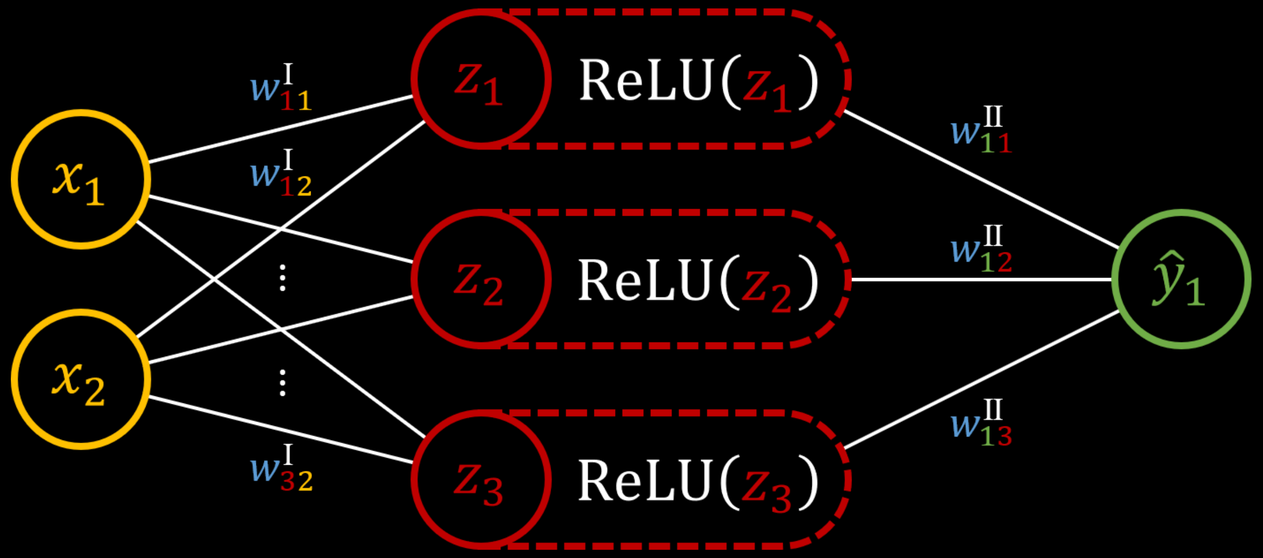
$$\hat{y}_1 = \mathrm{ReLU}\big(z_1(x)\big) \cdot w^{\mathrm{II}}_{11} + \mathrm{ReLU}\big(z_2(x)\big) \cdot w^{\mathrm{II}}_{12} + \mathrm{ReLU}\big(z_3(x)\big) \cdot w^{\mathrm{II}}_{13} + b^{\mathrm{II}}_1$$

$$= \mathrm{ReLU}(Z) \cdot \big(W^{\mathrm{II}}\big)^T + B^{\mathrm{II}}$$

$$= \mathrm{ReLU}\left(\left[X \cdot \big(W^{\mathrm{I}}\big)^T + B^{\mathrm{I}}\right]\right) \cdot \big(W^{\mathrm{II}}\big)^T$$

The diagram ignores the **bias** b for simplicity

DEEP NEURAL NETWORKS



$$\hat{y}_1 = \mathrm{ReLU}\left(\left[X \cdot \left(W^{\mathrm{I}}\right)^T + B^{\mathrm{I}}\right]\right) \cdot \left(W^{\mathrm{II}}\right)^T + B^{\mathrm{II}}$$

The diagram ignores the **bias** b for simplicity

DEEP NEURAL NETWORKS

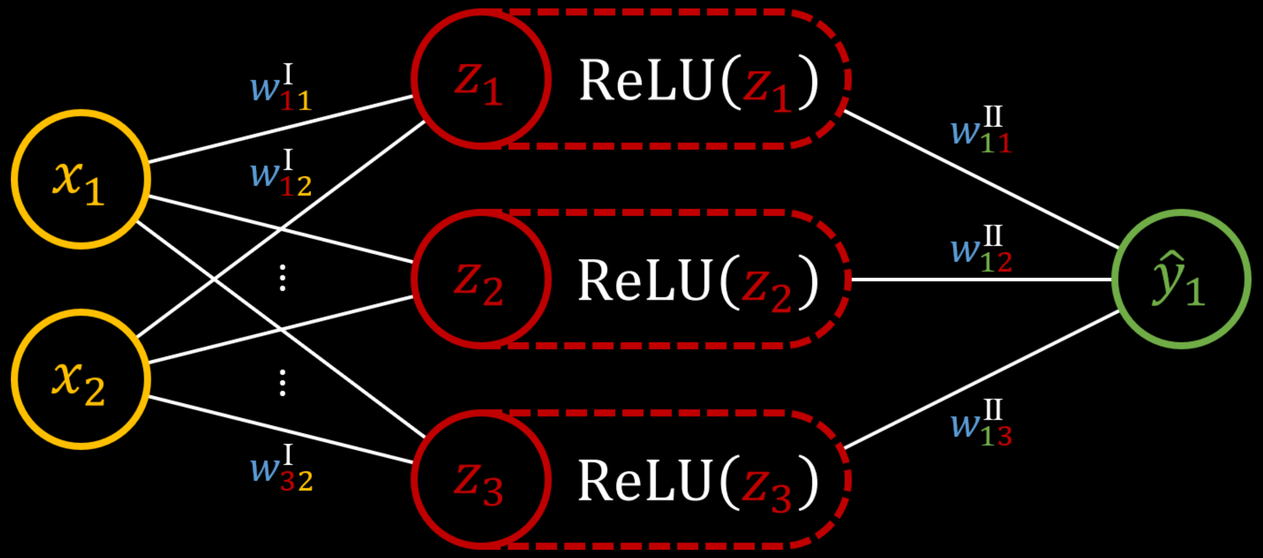
```python
from collections import OrderedDict
import torch.nn as nn

C = 2         # Number of input features (channels)
H = 3         # Number of hidden features
L = 1         # Dimension of output features (labels)

deep_model = nn.Sequential(
    # Use `OrderedDict` so we can adress layers by name.
    OrderedDict({
        'layer_I':    nn.Linear(C, H),
        'activation': nn.ReLU(),
        'layer_II':   nn.Linear(H, L)
    })
)

print(deep_model.layer_I.weight.shape)
print(deep_model.layer_I.bias.shape)
print(deep_model.layer_II.weight.shape)
print(deep_model.layer_II.bias.shape)
```

```
torch.Size([3, 2])
torch.Size([3])
torch.Size([1, 3])
torch.Size([1])
```



$$\hat{y}_1 = \mathrm{ReLU}\left(\left[X \cdot \left(W^{\mathrm{I}}\right)^T + B^{\mathrm{I}}\right]\right) \cdot \left(W^{\mathrm{II}}\right)^T + B^{\mathrm{II}}$$

The diagram ignores the **bias** b for simplicity

DEEP NEURAL NETWORKS

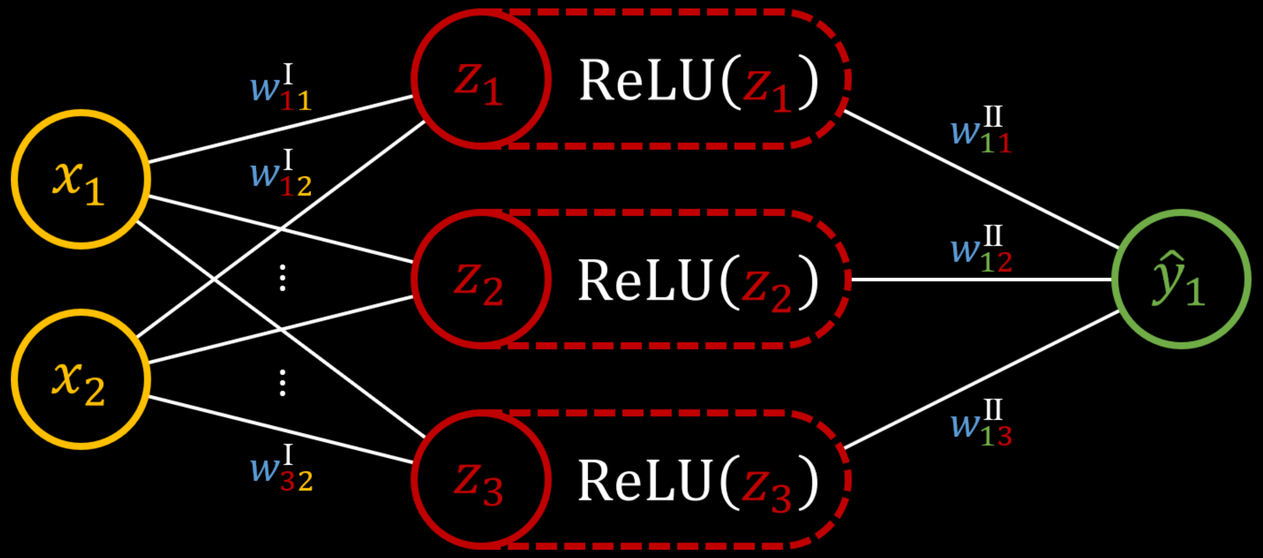
```python
from collections import OrderedDict
import torch.nn as nn

C = 2         # Number of input features (channels)
H = 3         # Number of hidden features
L = 1         # Dimension of output features (labels)

deep_model = nn.Sequential(
    # Use `OrderedDict` so we can adress layers by name.
    OrderedDict({
        'layer_I':    nn.Linear(C, H),
        'activation': nn.ReLU(),
        'layer_II':   nn.Linear(H, L)
    })
)

print(deep_model.layer_I.weight.shape)
print(deep_model.layer_I.bias.shape)
print(deep_model.layer_II.weight.shape)
print(deep_model.layer_II.bias.shape)
```

```
torch.Size([3, 2])
torch.Size([3])
torch.Size([1, 3])
torch.Size([1])
```



$$\hat{y}_1 = \mathrm{ReLU}\left(\left[X \cdot \left(W^{\mathrm{I}}\right)^T + B^{\mathrm{I}}\right]\right) \cdot \left(W^{\mathrm{II}}\right)^T + B^{\mathrm{II}}$$

- Shapes of **weight tensors** match the number of **input** and **output** features:
  - W^{I}: H **rows** and C **columns**
  - W^{II}: L **rows** and H **columns**

The diagram ignores the **bias** b for simplicity

DEEP NEURAL NETWORKS: SUMMARY

- We want to learn the **weights** w and **biases** b such that $\hat{y} \approx y$
- Each linear layer has its own **learnable tensors** for **weight** W and **bias** B
- The computation for every linear layer is described by

$$\langle \text{layer_output} \rangle = \langle \text{layer_input} \rangle \cdot (W)^T + B$$

- PyTorch organizes the shape of **weight tensor** W by the layer outputs as `weight.shape = torch.Size([out_features, in_features])`
  - The **number of rows** corresponds to the **number of the layer's output features**
  - The **number of columns** corresponds to the **number of the layer's input features**
- PyTorch makes model definition really easy by abstracting most of these details away, but this knowledge is crucial when debugging models

OUTLOOK: NEXT WEEK

- The material for the next week will include more technical topics:
  - Data Pipelines
  - Data Transformation and Augmentation
  - Data Split in Training, Validation and Test Datasets
  - Modular & Dynamic Model Design
  - Monitoring and Logging

- **Assignments:** Who will present next?

- **Bonus:**
  Think about why image classification for ice cream flavors might not be a good task for CNNs?